Routing in React with React Router.

Introduction

In this lesson, we will explore how to set up routing in a React application using **React Router**. Routing allows us to create a single-page application (SPA), where different components can be displayed based on the URL path without reloading the entire page. We will cover the following concepts:

1. **What is React Router?**
2. **Setting Up Routing in React Router**
3. **Creating Routes for Components**
4. **Navigating Between Pages**

1. What is React Router?

React Router is a standard library for routing in React applications. It enables you to handle navigation, render different components based on the URL, and manage navigation state. With React Router, you can create an app with multiple pages, where each page corresponds to a unique URL path, but everything is rendered dynamically within the same page, providing a smooth user experience.

2. Setting Up React Router

To use **React Router** in your application, you first need to install it:

sudo npm install react-router-dom@latest

Once installed, you can begin creating routes in your app.

3. Creating Routes for Components

In a React application, routes are defined using **Route** components.

We already have a simple app that includes three components:

- **FirstComponent**
- **SecondComponent**
- **ThirdComponent**

And we will navigate between these components using links in the app's navigation menu.

4. Understanding React Router Components

Before we dive into the code, let's first understand the key components that React Router v7 provides for handling routing:

1. **<Router>**: This is the root component that wraps your entire routing structure. It provides routing context to your application.
2. **<Routes>**: This component holds all of your route definitions. It is the container for <Route> components.
3. **<Route>**: A route maps a URL path to a specific React component. It contains the path (the URL pattern) and the element (the React component to render).
4. **<Link>**: This component is used for navigation. It allows you to navigate between routes without causing a full page reload.

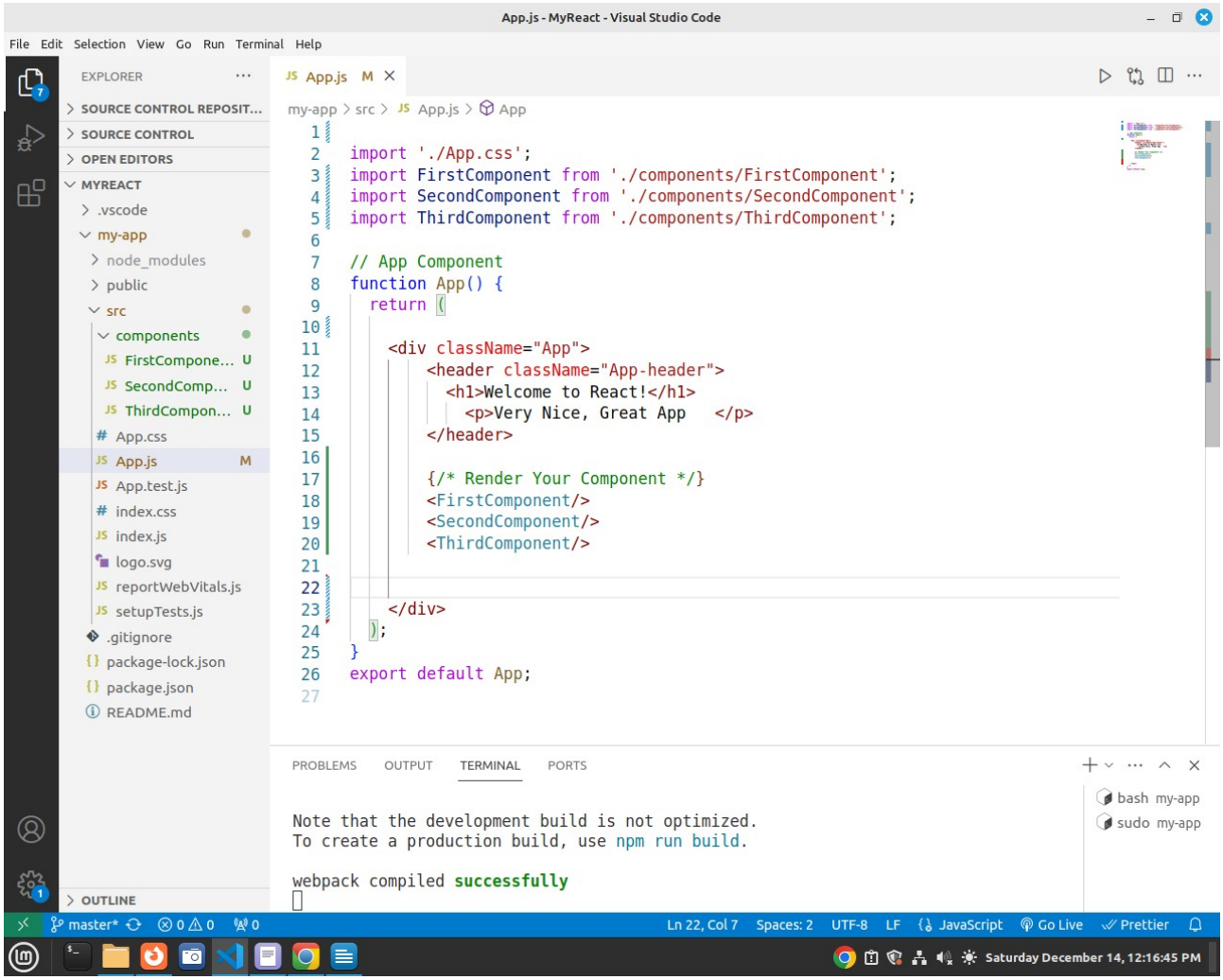
5. Setting Up Routing in App.js

Here's a step-by-step breakdown of how to set up routing in your app using the following structure:

- **FirstComponent**, **SecondComponent**, and **ThirdComponent** are the components we will display when the user navigates to /first, /second, and /third respectively.

- We will use the **Link** component for navigation in the navigation menu.

Below Is the current look of our App.js



In Above code, Since we want to use the Routing Technical to access our components,

Remove the lines 18,19,20

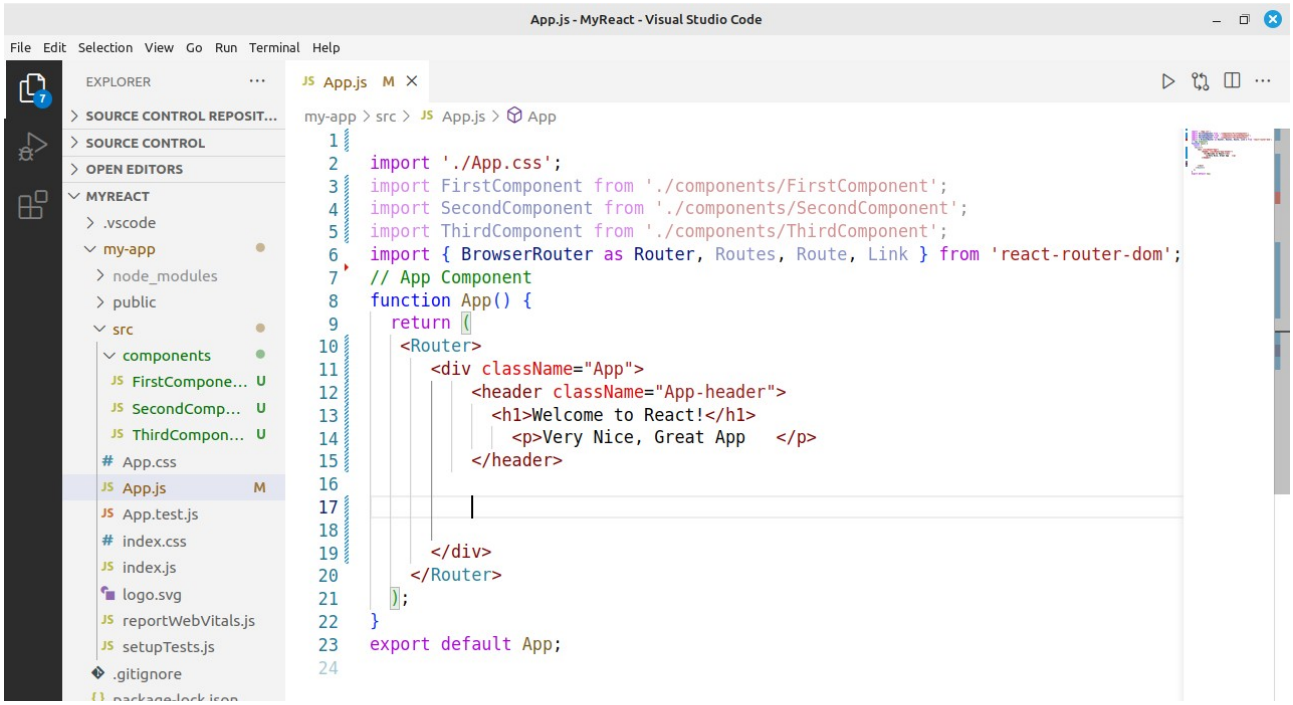
In below code we will set up a React Router.

We import necessary dependencies.

```
//Imports

import { BrowserRouter as Router, Routes, Route, Link } from 'react-router-dom';
```

Then Wrap the container App Div with Router Component.



The **<Router>** component is the root container that wraps the entire routing structure of your application. It listens for changes in the URL and decides which components to display based on the current path.

6. Routes and Route Components

```
{/* Below we create routes that will link to specific Components */}
<Routes>
  <Route path="/first" element={<FirstComponent />} />
  <Route path="/second" element={<SecondComponent />} />
  <Route path="/third" element={<ThirdComponent/>} />
</Routes>
```

MODCOM Institute of Technology

<Routes> is the container for all your routes.

- **<Route>** maps a path to a component. Each route has:
- **path**: The URL pattern (e.g., /component1).
- **element**: The React component to render when the path matches the URL (e.g., <**FirstComponent** />).

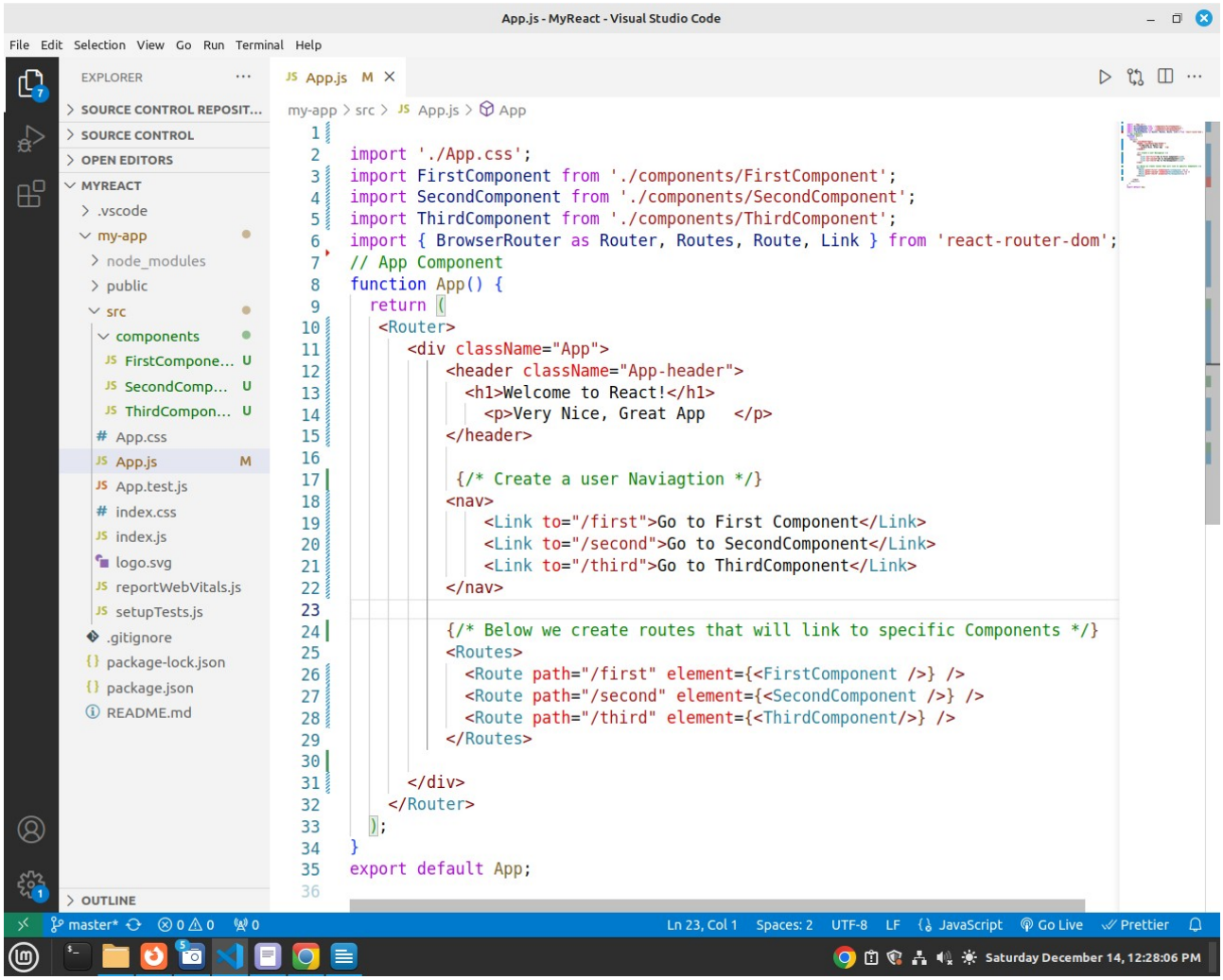
7. Link Components for Navigation

```
{/* Create a user Naviagtion */}
<nav>
    <Link to="/first">Go to First Component</Link>
    <Link to="/second">Go to SecondComponent</Link>
    <Link to="/third">Go to ThirdComponent</Link>
</nav>
```

<Link> is used to navigate between routes without causing a full page reload. Each **<Link>** has a to prop that specifies the path to navigate to.

- **to="/fist"** will navigate to /first and render **FirstComponent**
- **to="/second"** will navigate to /second and render **SecondComponent**
- **to="/third"** will navigate to /third and render **ThirdComponent**.

Next is the Full code for Your **App.js**



8. Testing the Application

Now, when you run the application (npm start), you will see:

1. A navigation menu with links to **FirstComponent**, **SecondComponent**, and **ThirdComponent**.
2. Clicking on each link will display the corresponding component below the navigation menu.

9. How the App Works:

- **Initial Page Load**: The app will load with the navigation menu and a default page (you can add a default route if needed).

- **Navigating to Routes**: Clicking on a link updates the URL and renders the corresponding component.
- **No Page Reload**: React Router takes care of the URL changes without causing a page reload, making the app feel like a Single Page Application (SPA).

10. Conclusion

In this chapter, we have:

- Set up routing in a React app using **React Router**.
- Created three components (**FirstComponent**, **SecondComponent**, and **ThirdComponent**) and set up routes for them.
- Used **<Link>** for navigation and **<Routes>** with **<Route>** to handle path-to-component mapping.

React Router allows us to build complex, navigable SPAs where different components are rendered based on the URL, all while maintaining a smooth user experience. By using the new API in React Router, routing has become more intuitive and easier to manage

You can style your Links by giving class .**navlink** and style it in App.css

```
<nav>

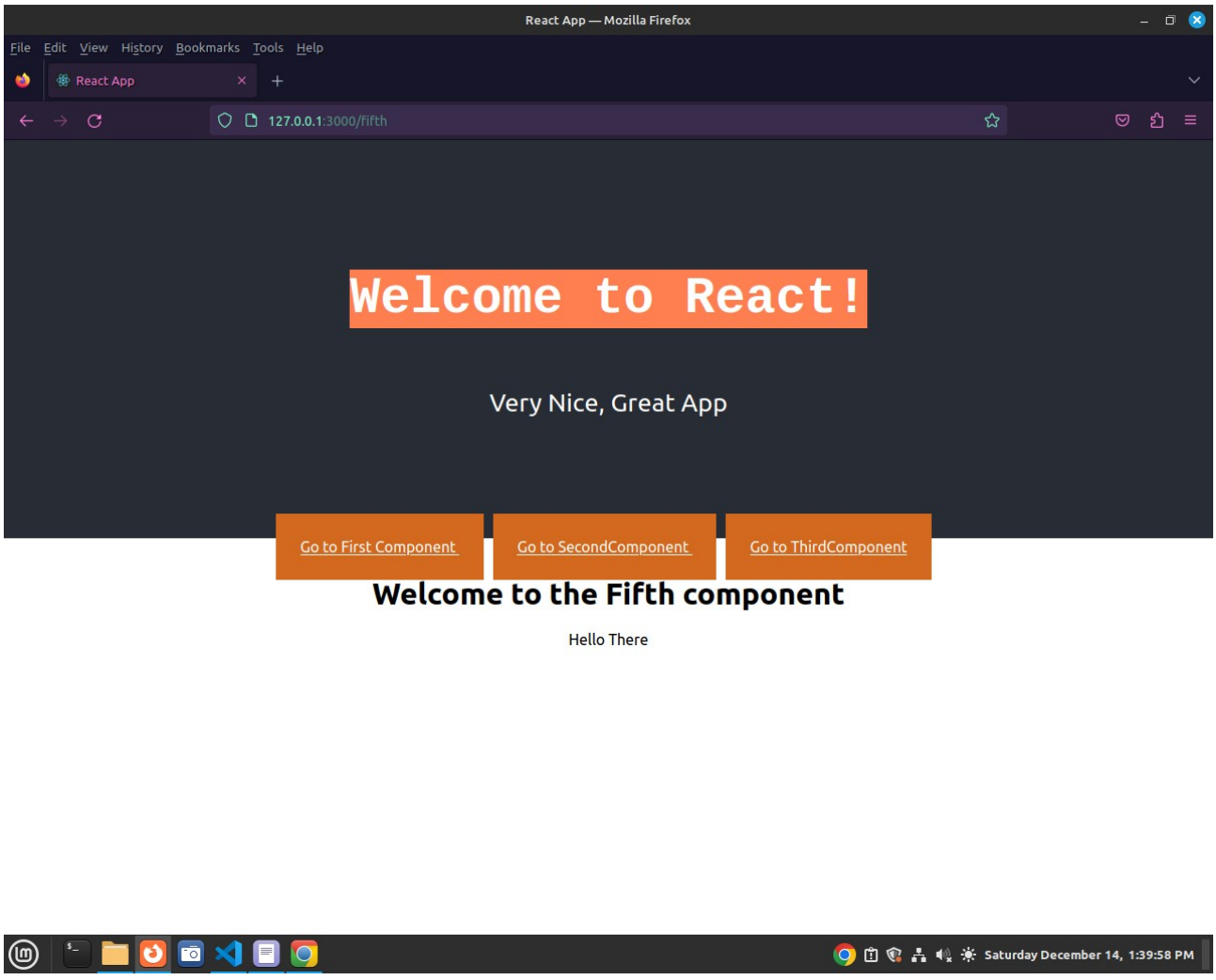
<Link to="/first" className='navinks>Go to First Component </Link>
<Link to="/second" className='navlinks'>Go to SecondComponent </Link>
<Link to="/third" className='navlinks'>Go to ThirdComponent </Link>
</nav>
```

in src/App.css

```
.navlinks{

margin-right: 10px;
background-color: chocolate;
padding: 2%;
color: white;
}
```

Output



Student Task:

In **components** Folder create a Component named **FourthComponent.js,** Add a Basic JSX code inside **FourthComponent.js, (A div, heading and a paragraph).**

Add a Route to this **FourthComponent.js** in **App.js**

Test your Navigation to **FourthComponent.js** by running your React App.